

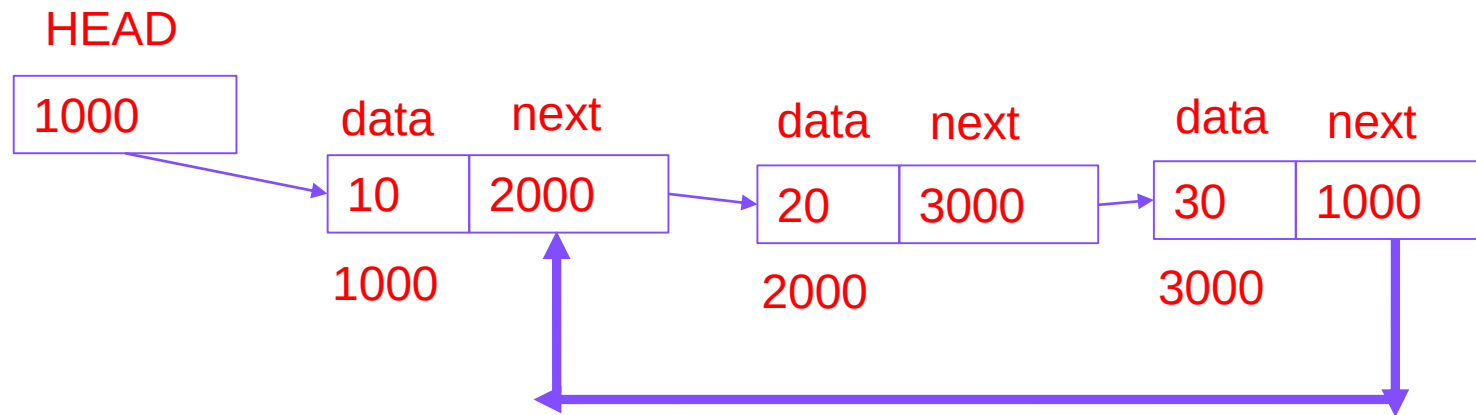
DATA STRUCTURES

Circular Linked List

Module 2

Circular Linked List: Insertion

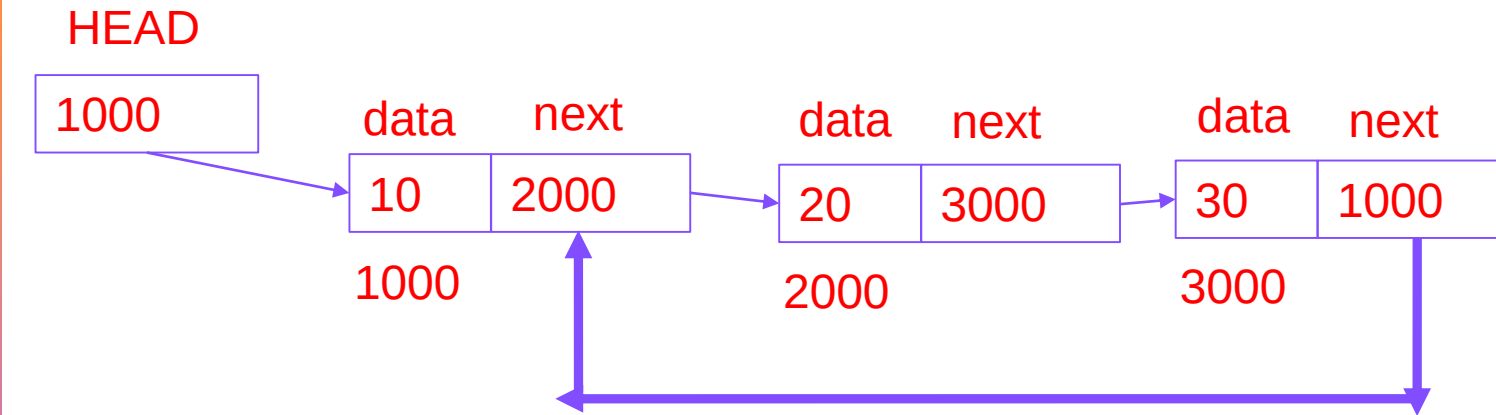
- 1. At the front of the linked list
- 2. After a given node is same as singly linked list
- 3. At the end of the linked list



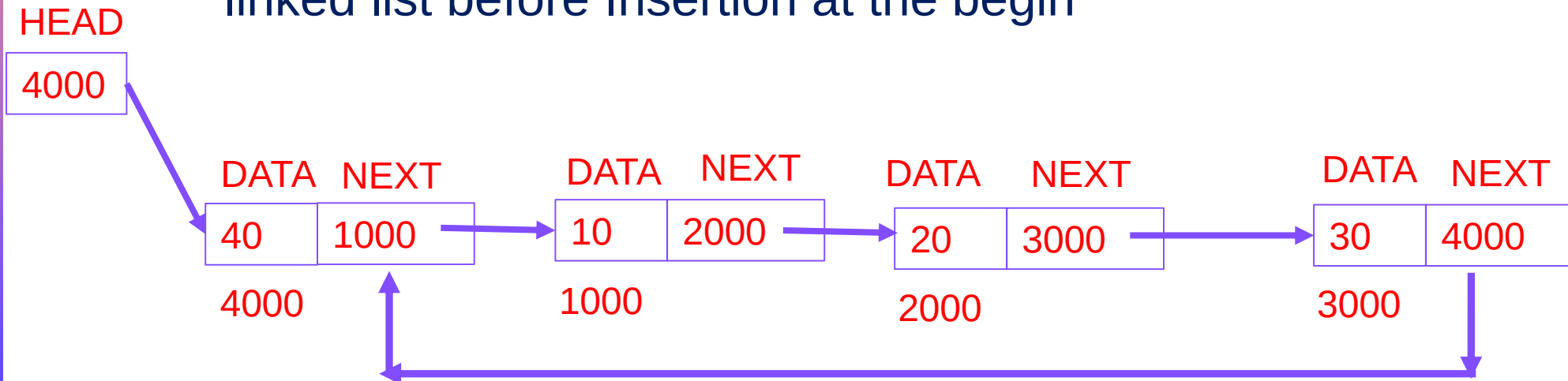
Circular linked list

Insertion-at the begin

```
// A linked list node
struct Node
{
    int data;
    struct Node *next;
};
```



linked list before Insertion at the begin



linked list after Insertion at the begin

Insertion-at the begin

STEP 1. CREATE A NEW NODE(NEWNODE)
NEWNODE → DATA := VALUE
NEWNODE → NEXT := NULL

STEP 2: LOCATE THE LAST NODE
SET PTR := HEAD
WHILE PTR→NEXT ≠ HEAD DO
PTR := PTR → NEXT.
DONE

CREATING A NEW NODE

1

NEWNODE

4000

DATA NEXT

40

NULL

4000

2

ITER	PTR	PTR→NEXT ≠ HEAD	PTR := PTR → NEXT
1	1000	2000 ≠ 1000 True	PTR := 2000
2	2000	3000 ≠ 1000 True	PTR := 3000
3	3000	1000 ≠ 1000 False	-

2

HEAD

1000

data next

10

2000

1000

data next

20

3000

2000

data next

30

1000

3000

PTR

3000

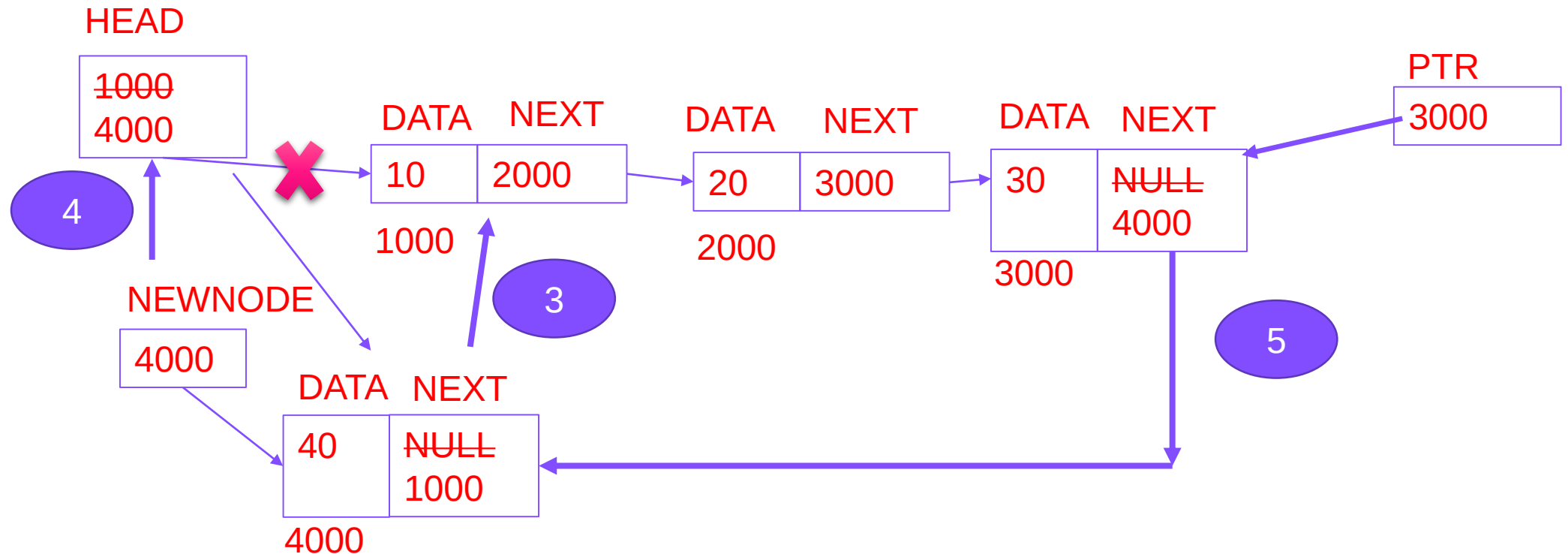
Locating the last node

Insertion-at the begin

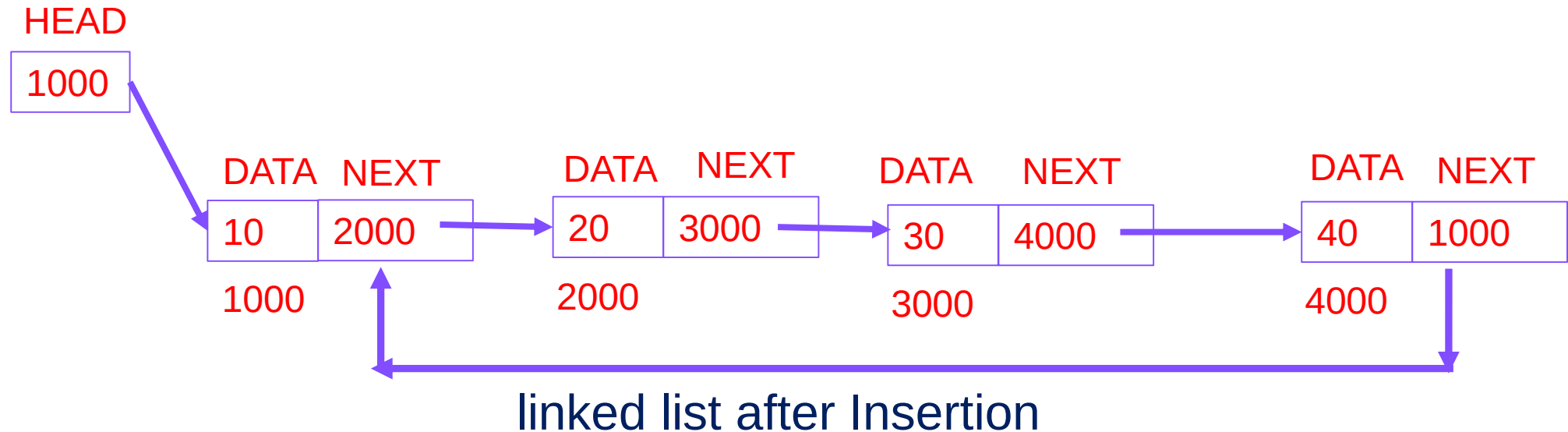
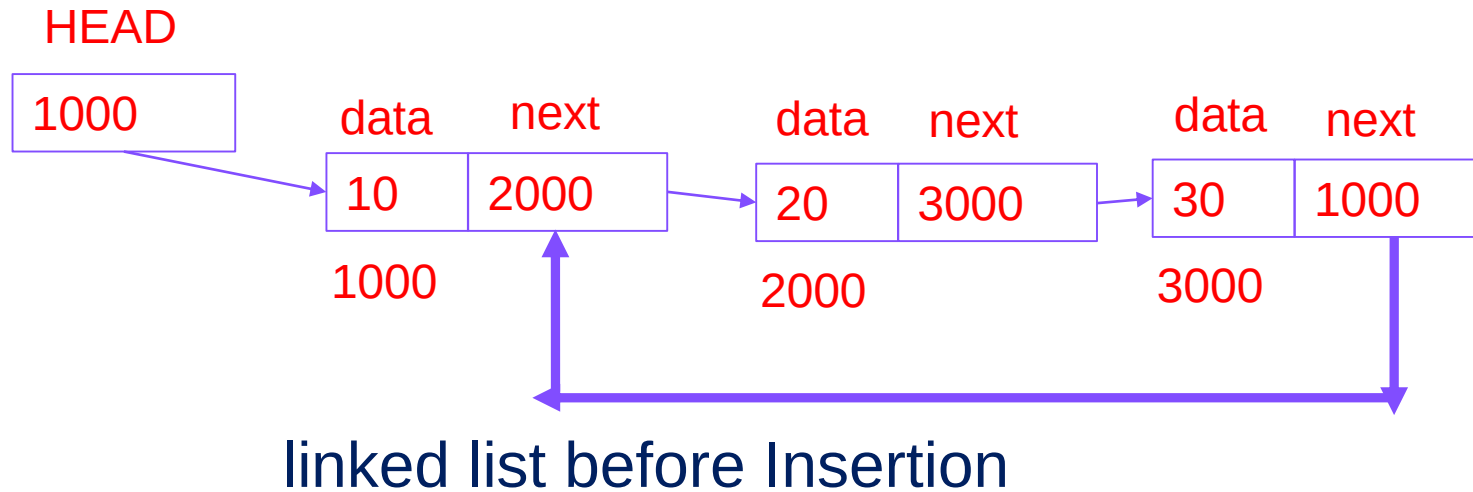
STEP 3: SET NEWNODE → NEXT := HEAD [Copy first node address pointed by HEAD into New node's Address field]

STEP 4: HEAD := NEWNODE [Copy new node address into HEAD]

STEP 5: PTR → NEXT := HEAD [Set the last node address filed to new HEAD.]



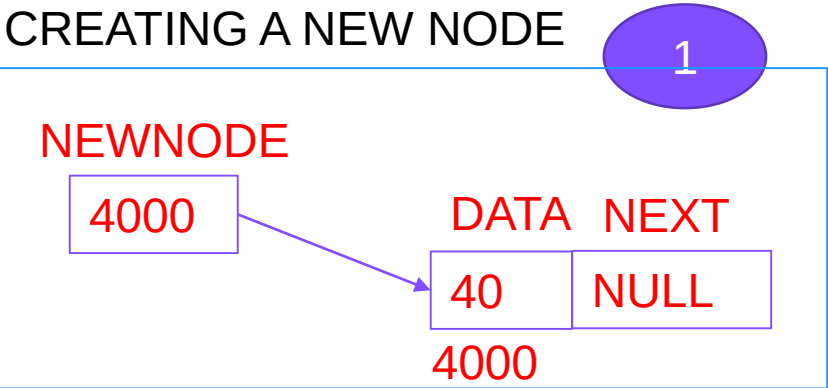
Insertion-at the end



Insertion-at the end

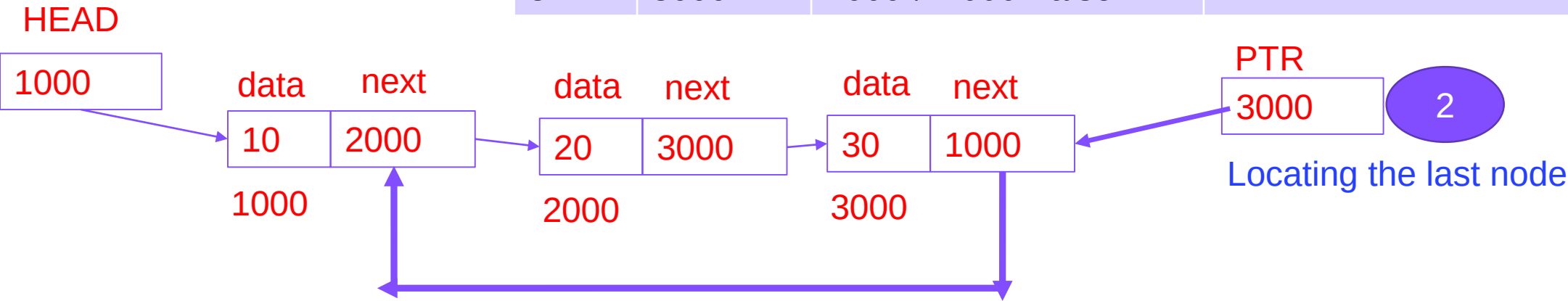
STEP 1. CREATE A NEW NODE(NEWNODE)
NEWNODE → DATA := VALUE
NEWNODE → NEXT := NULL

STEP 2: LOCATE THE LAST NODE
SET PTR := HEAD
WHILE PTR→NEXT ≠ HEAD DO
PTR := PTR → NEXT.
DONE

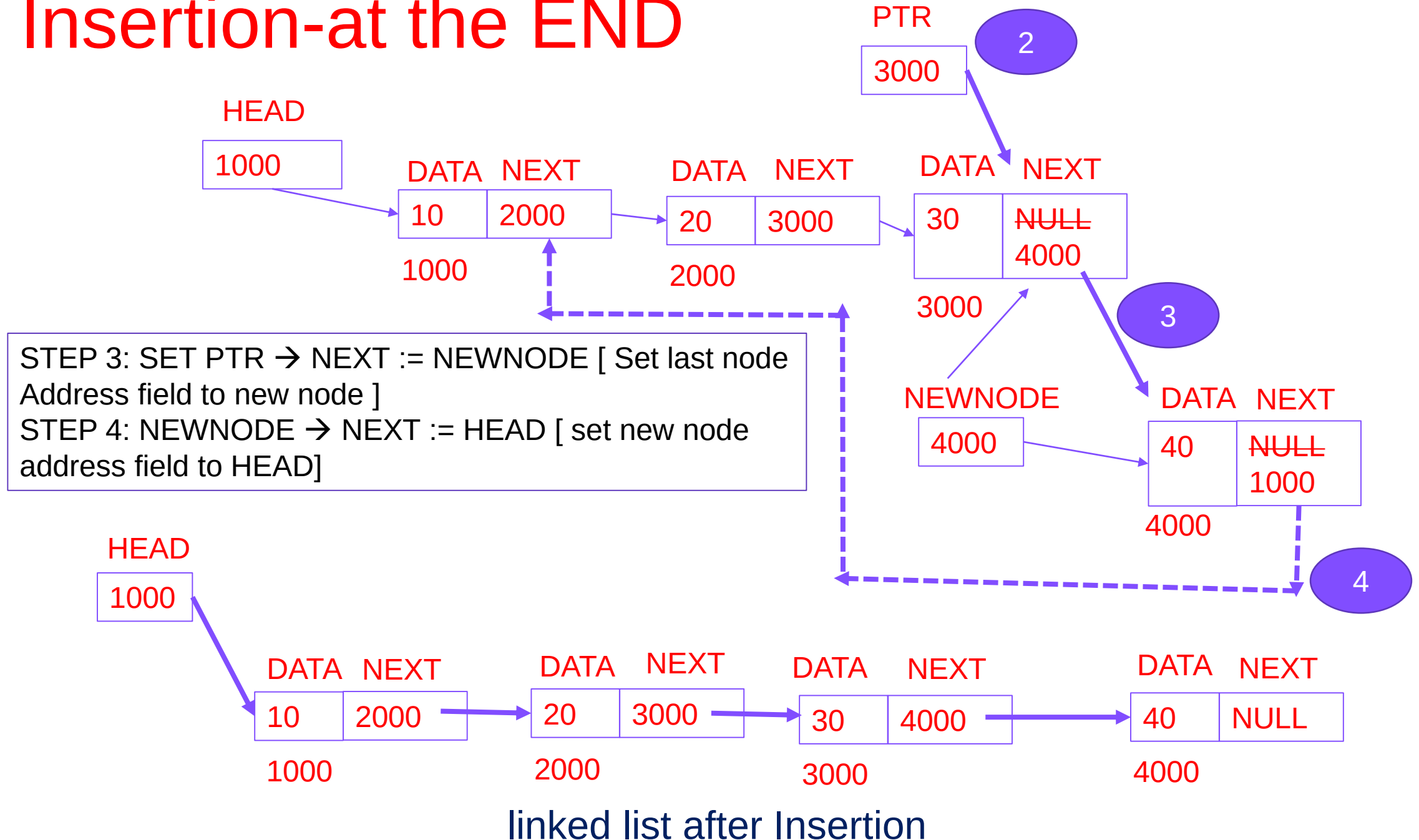


2

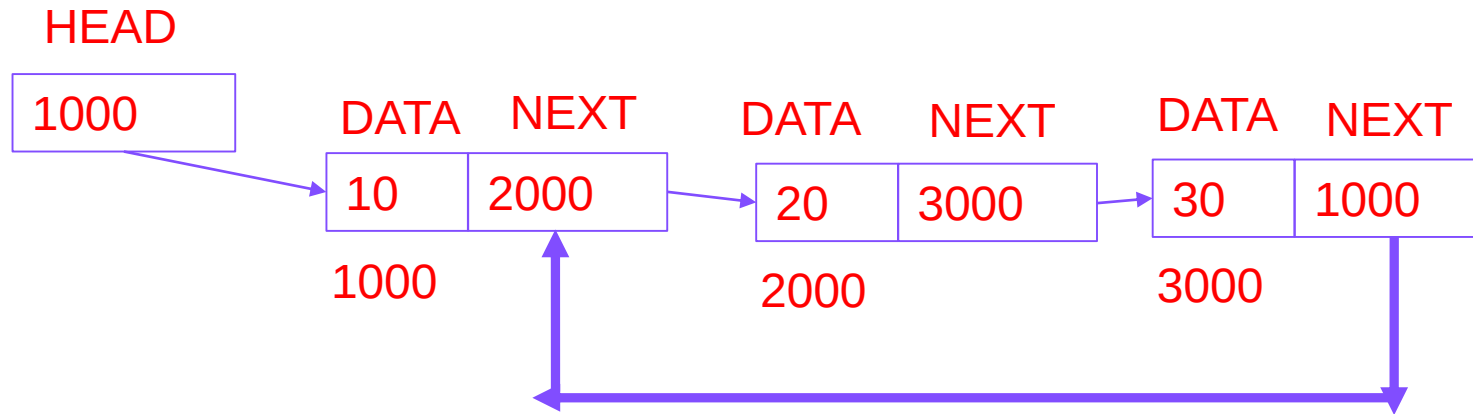
ITER	PTR	PTR→NEXT ≠ HEAD	PTR := PTR → NEXT
1	1000	2000 ≠ 1000 True	PTR := 2000
2	2000	3000 ≠ 1000 True	PTR := 3000
3	3000	1000 ≠ 1000 False	-



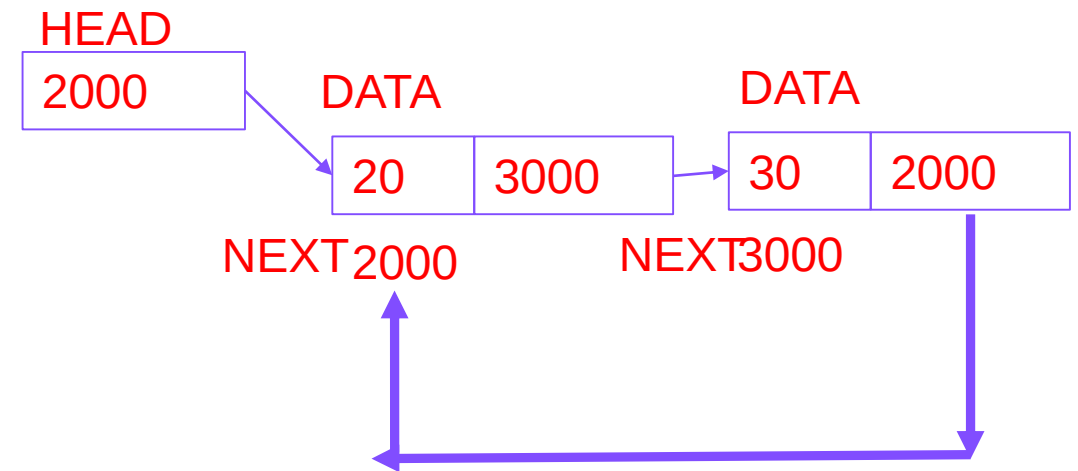
Insertion-at the END



Deletion at the begin-from the linked list



linked list before Deletion



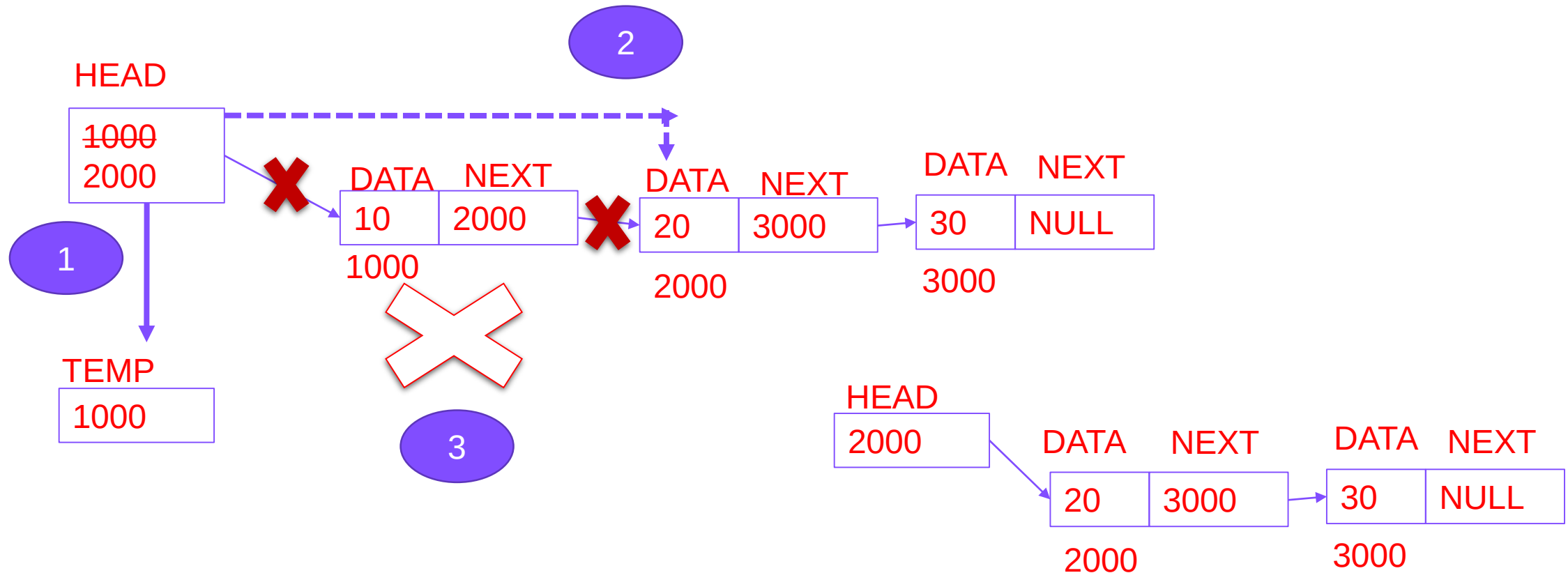
linked list after deletion

Deletion-at the begin

STEP 1. $TEMP := HEAD$ [Preserve the first node address into TEMP]

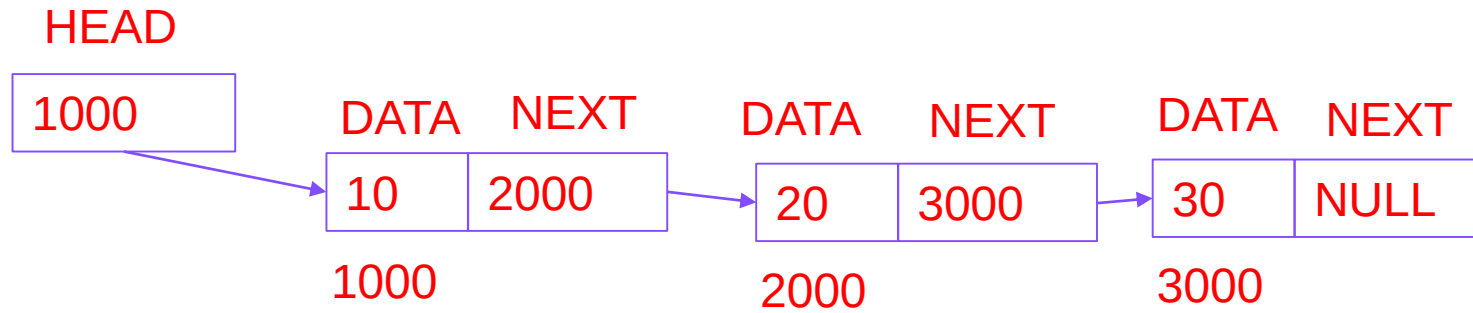
STEP 2. $SET\ HEAD \rightarrow HEAD := NEXT$ [Copy second node address into HEAD]

STEP 3. $FREE(HEAD)$ [Delete the first node, address available at TEMP]

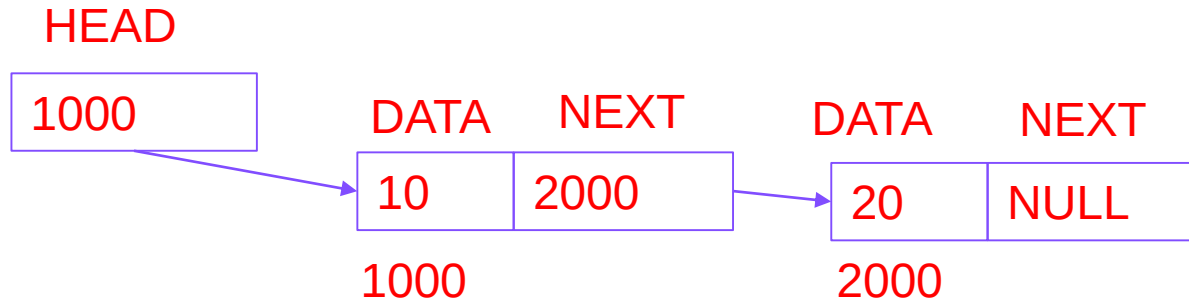


linked list deletion from the begin

Deletion at the end-from the linked list



linked list before Deletion at the end



linked list after Deletion at the end

Deletion-at the end

STEP 1. LOCATE THE LAST NODE (TEMP) and LAST BEFORE node (PREV)

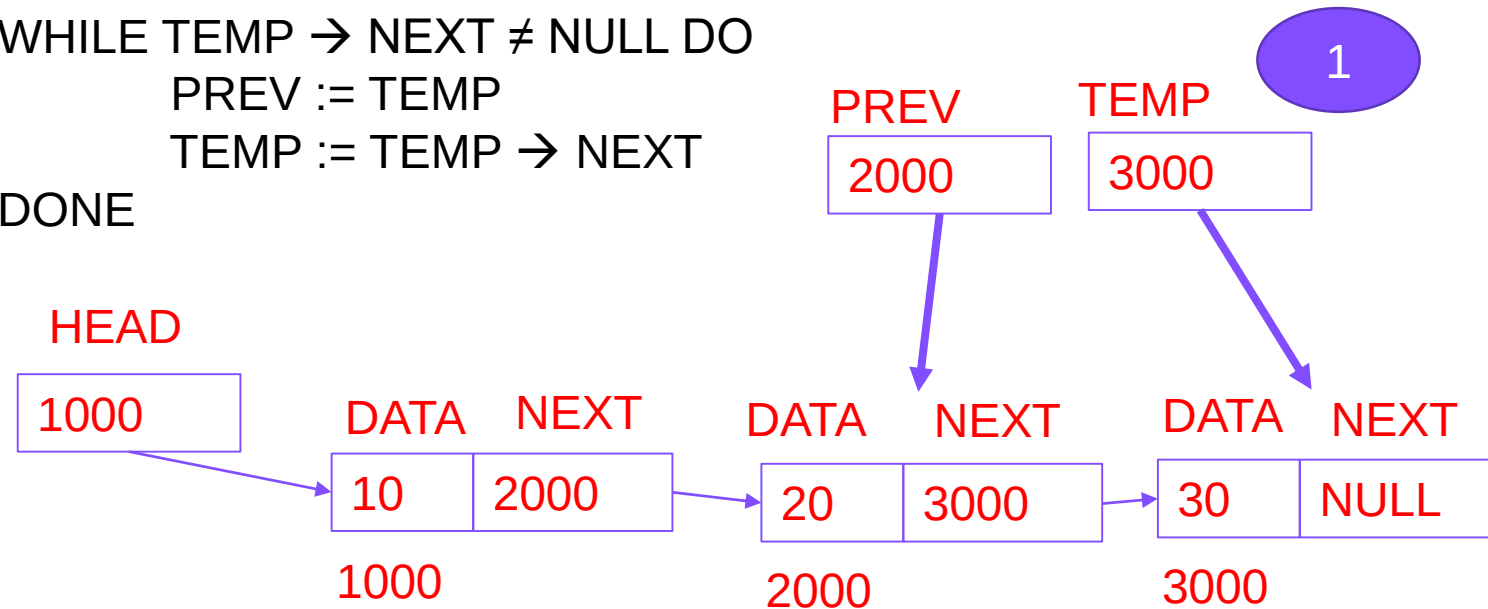
TEMP:=HEAD

WHILE TEMP → NEXT ≠ NULL DO

PREV := TEMP

TEMP := TEMP → NEXT

DONE

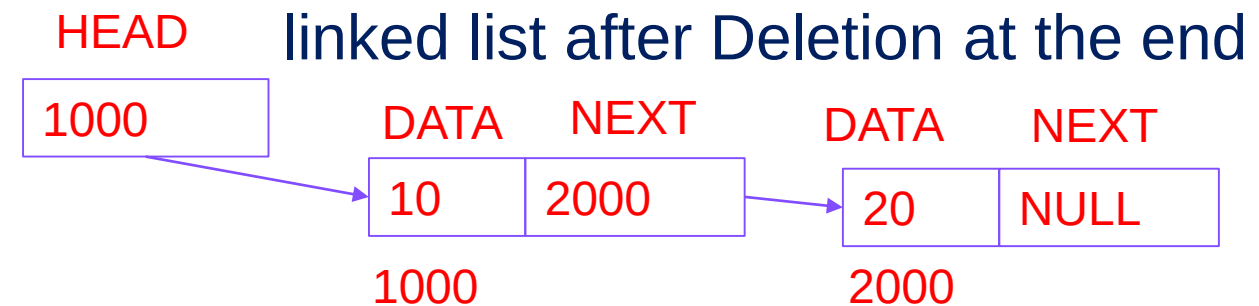
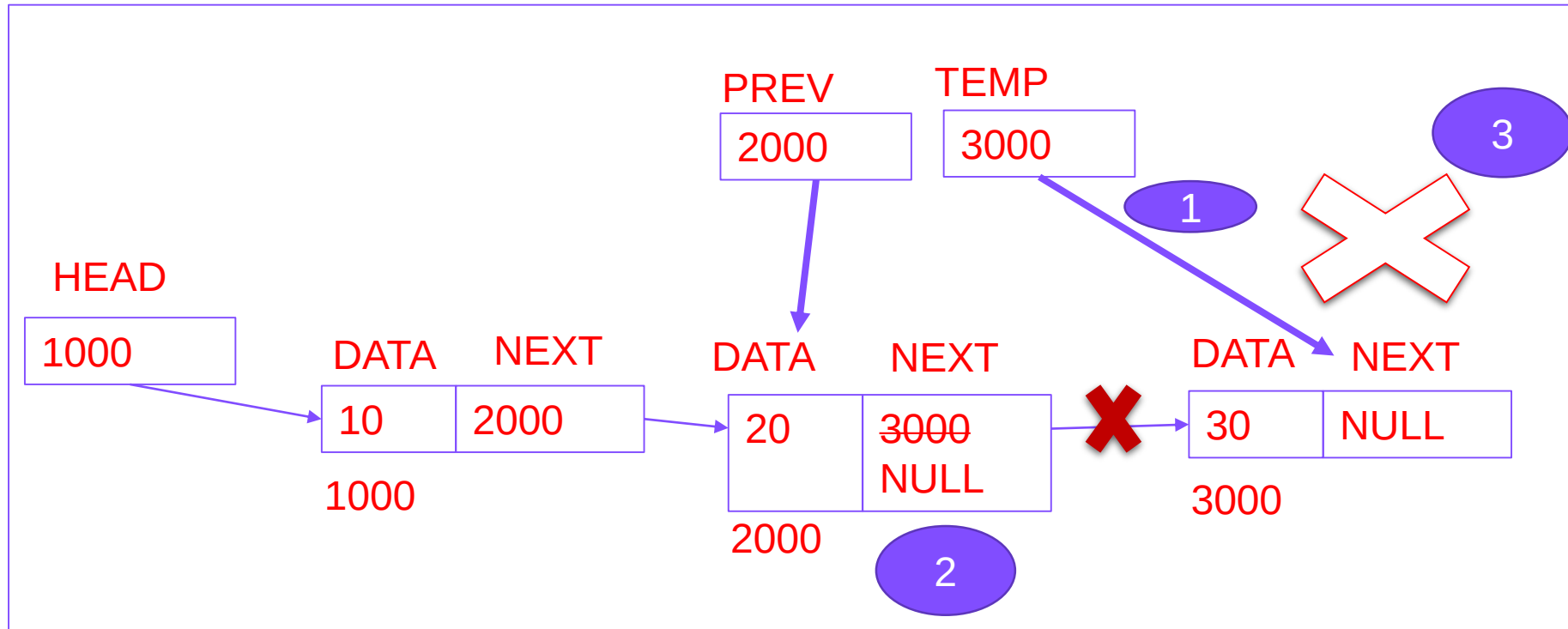


ITER	TEMP	TEMP→NEXT ≠ NULL	PREV:=TEMP	TEMP := TEMP → NEXT
1	1000	2000 ≠ NULL True	1000	TEMP := 2000
2	2000	3000 ≠ NULL True	2000	TEMP := 3000
3	3000	NULL ≠ NULL False	-	-

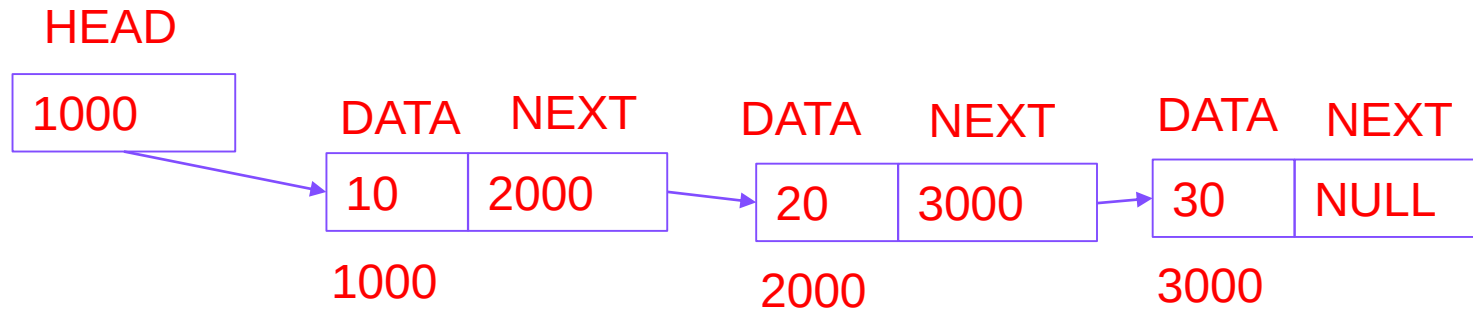
Deletion-at the end

STEP 2: PREV → NEXT:= NULL. [SET the address of last before node to NULL]

STEP 3: FREE (TEMP) // Delete the last node whose address is available in TEMP.

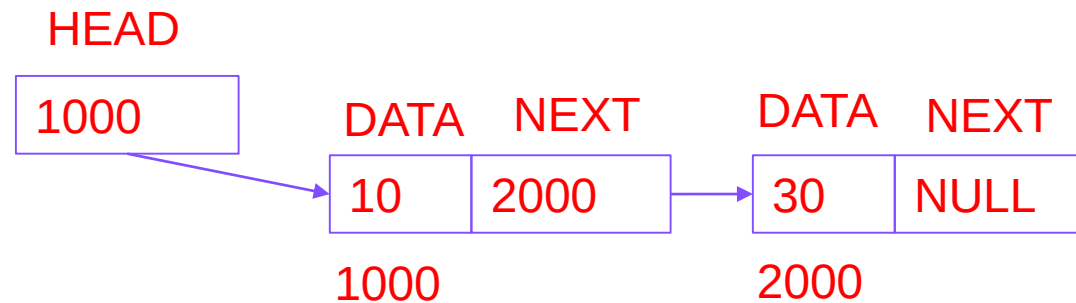


Deletion-after a given node (Nth node)



linked list before Insertion

```
// A linked list node
struct Node
{
    int data;
    struct Node *next;
};
```



linked list after Insertion: Given node number is 1

Deletion-after a given node (Nth node)

STEP 1: LOCATE Nth Node and Its previous node

SET TEMP:= HEAD

SET I := 1

WHILE TEMP ≠ NULL

BEGIN

IF I = N THEN

BREAK

END IF

PREV := TEMP.

TEMP := TEMP → NEXT.

END [TEMP contains the address of Nth Node and PREV contains preceding node's address]

STEP 2: IF TEMP → NEXT == NULL THEN // If N is the last node.

PREV → NEXT := NULL // set previous node address field to NULL.

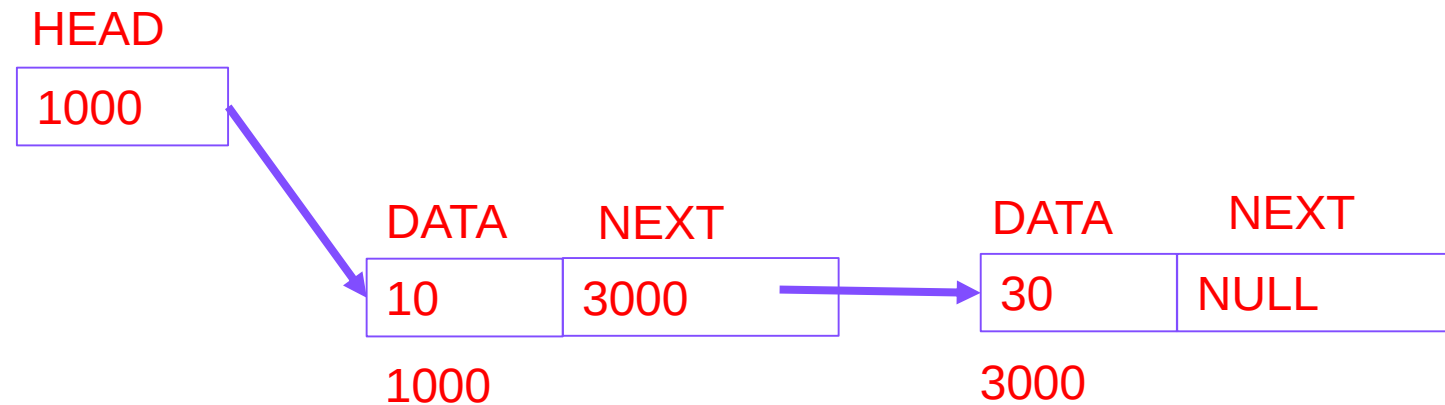
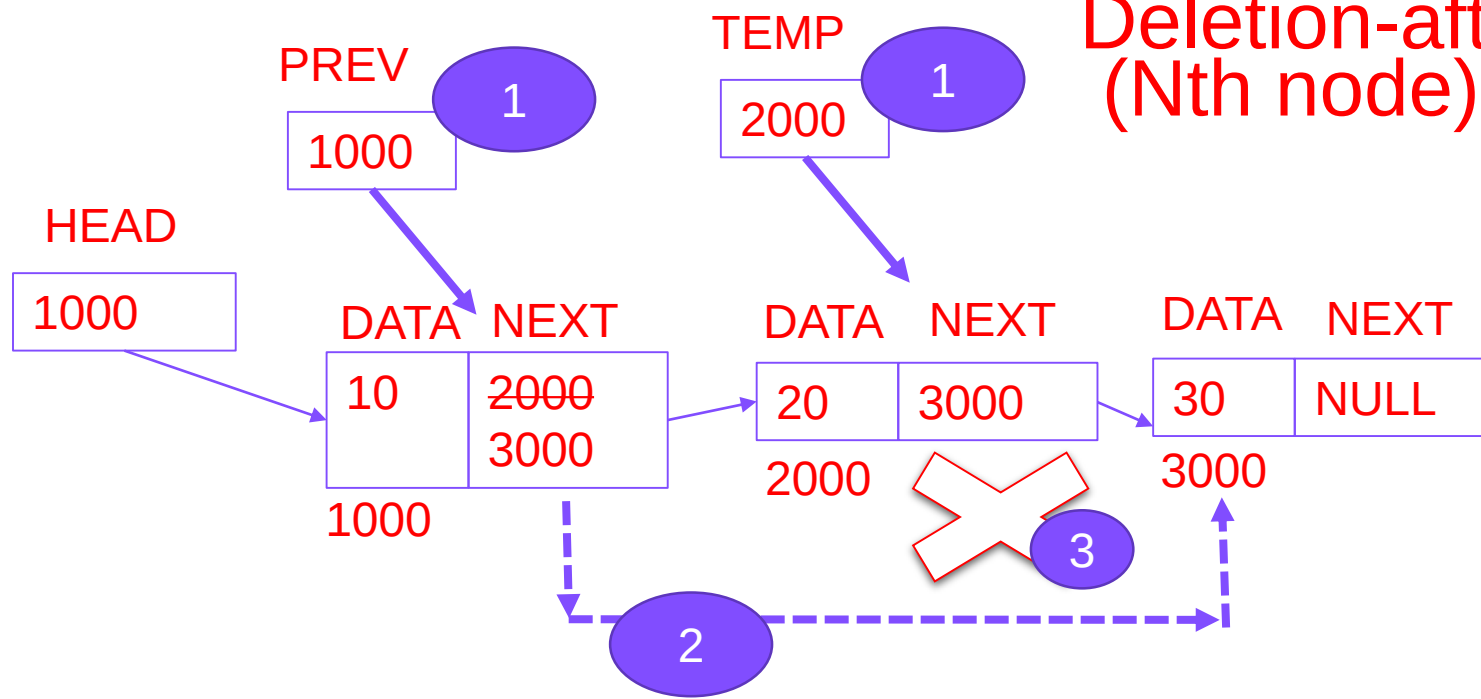
FREE (TEMP). // Delete the last node whose address is available in TEMP.

ELSE THEN // If N is not a last node

PREV → NEXT := TEMP → NEXT [Set previous node of Nth node's address field to
successor node's address]

STEP 3: FREE (TEMP) [Deletion of a node TEMP]

Deletion-after a given node (Nth node)



linked list after Deletion